Notice that we pass a string literal of `'2'` into the function `increment()`; we're not declaring `strict_types=1`, so PHP converts the incoming string to a number.

As we can see, declaring base types is straightforward. I've deliberately avoided the standalone pseudo-types, such as `mixed`, for now, but we will return to them later on. Things get a little more complex with composite types.

Composite Types

All composite types are combinations of base types, with rules for defining how you need the argument or return data to satisfy the declaration. You might require that a type matches one of a set, all of a set, or possibly one or the other (i.e., "either match this single type or this group of types"). Let's start with the simplest case.

Nullable Types

Added in PHP 7.1, by prefixing the type name with a question mark character (e.g., `?string`, `?SomeClass`), the declaration's meaning becomes "this type or null." To declare that a function returns a `string` or `null`, we can use `function someFunction(): ?string` like this:

```
declare(strict_types=1);

function doSomething(?string $in): ?string
{
    return ($in === null) ? null : 'Not null';
}

var_dump(doSomething('a string'));
var_dump(doSomething(null));

Output:
string(8) "Not null"
NULL
```

The code above should need little explanation if you've followed the rest of the chapter so far; it's running in strict mode to demonstrate that we're satisfying all of the type declarations for `doSomething()`. PHP has a weird quirk with nullable types, which we'll see in a moment.